

Tailoring Overview

Introduction

Tailoring is the process of adapting, or tailoring, the standard IFS Cloud solutions to fit even better for a specific business. It typically involves performing configurations (e.g. of the user interface or process automation), developing reports and integrations, and where motivated also perform customizations to extend or change aspects of the solution.

The objective of this guide is to provide an understanding of the tailoring options available, as well as options are suitable for different situations and wanted outcomes. A good understanding of this will result in a better experience, involving less work and cost, of applying future Releases to a tailored solution.

IFS Cloud offers three ways that tailoring can be done:

- **Configuration:** use consultant and power-user friendly designers and tooling to extend the data model, change the user interface, automate processes and much more.
- **Extend on the inside / Customize:** customize using the IFS Cloud platform in order to add, extend or override functionality. This is done using our low-code development environment and Layered Application Architecture (LAA) to separate customizations from the core application. But you must never customize anything belongs to a Framework component as this could impair the appropriate function of IFS Cloud. In 22R1 and onwards LAA or Extend on the inside is restricted for Framework components. Please refer to [restrict LAA/Extend on the inside](#) for more details.
- **Extend on the outside:** using low code development platforms like PowerApps and Mendix, integration platforms like Boomi and Snaplogic, or just develop custom clients and integrations using the technology of your choice.

In addition to the tailoring types that talk about how tailoring is done, it is also important to understand the tailoring levels and how they impacts the experience of updating to future releases. A higher tailoring level is more intrusive and in general results in a less automated, more labor intensive, experience of applying new releases. The tailoring types and tailoring levels are further explained and illustrated [below](#).

In order to strike the best balance between the business outcomes wanted, and the impact on the release update experience, make sure you have an understanding of the guidance on choosing the right type and level of tailoring.

Tailoring types vs CRIMs

Customers and implementation projects commonly talk about **C**onfigurations, **R**eports, **I**ntegrations and **M**odifications (CRIMs). Those terms are related to the wanted solution outcome, e.g. a particular report is wanted, or an specific integration interface to another system. That is different from the tailoring types that discuss the method, or HOW, a such wanted outcomes can be achieved. For example some integrations will be possible to do using configuration, where others might require extending on the inside (customize), some modifications might be possible to achieve by configuring a workflow etc.

There is often a one-to-many relationship in that one "CRIM" is realized by a combination of multiple tailoring items. For example one "configuration" to "add a new field on this page" might in reality consist of multiple configuration items such as a configured attribute for an entity, a workflow to validate values of that attribute, and a page designs to place the field onto the page. Or a "report" might consist of a customization to collect the report data, configuration for the report layout, and a page design to add a button that orders the report.

Handling tailoring's through the lifecycle

Tailoring's done with extend on the inside (customize) are managed using IFS Lifecycle Experience. They are stored in the customer solution repository and made available to development and QA environments within the provided build place. They are promoted to use place environments through deliveries.

Tailoring's made with configuration should also be exported and stored in the customer solution repository for safe keeping. From there they can optionally be included in deliveries, or managed through the lifecycle by export/import e.g. in the form of application configuration packages.

Tailoring done with extend on the outside fall outside the scope of what is managed in IFS Lifecycle Experience.

Choosing the right type and level of tailoring

Some desired outcomes or functional requirements can only be met using a particular type of tailoring. But in many situations a choice is possible. For example, virtually everything that can be done through configuration (e.g. change UI layouts, extend the data model) can also be done by extending on the inside. And some, but not all, of what is possible to do with extending on the inside can also be done by extending on the outside.

The below table gives some high-level guidance. In general, you also want to keep the tailoring level as low as possible, but not to the point that you try to shoehorn something into a particular type or level of tailoring. Sometimes the right thing to do is to combine tailoring types to meet the full tailoring requirements for the customer solution.

	Extend on the outside	Configure	Extend on the inside (Customize)
Use with tailoring level(s)	Core, Configured, Customized	Configured, Customized	Customized - to add new functionality - to change functionality
When to use	Building apps / flows / automation that interact with multiple applications, where IFS Cloud is one. Surfacing functionality through a 3rd party UX or application. It is preferable that the added functionality is kept as independent as possible of IFS Cloud.	Changes are relatively small, or temporary. It is preferable that the work can be done by power users or application consultants.	Adding major capabilities consisting of several objects, substantial business logic, user interface. It is preferable that the user experience and/or tech stack of the added functionality is the same as that of IFS Cloud. There is a requirement to override existing business logic. Highest level of governance and traceability of origin and changes to the tailoring is wanted The requirements call for changes that are not possible to do with configuration

	Extend on the outside	Configure	Extend on the inside (Customize)
Examples	Task apps, Office add-ins, ESB / iPaaS, RPA	Page layouts, Report layouts, Data model extensions, Workflows, navigation flows, integrations	Data models, application logic, APIs, user experience
Who does it?	Integration Developer	Power user Application consultant	Developers
Tools	3 rd party app/workflow builders such as PowerApps, Outsystems, Mendix Integration platforms such as Boomi, LogicApps, SnapLogic, WebMethods. Development platforms and languages.	Designers—Lobby Designer, Page Designer, Workflow Designer etc. Configuration tools (e.g. Custom Entities, Custom Attributes, Custom Events) in IFS Solution Manager.	IFS Developer Studio
Where are changes done?	Outside IFS Cloud, tested against Build Place or Use Place	Build Place or Use Place	Build Place
Where are they stored?	N/A - outside the scope of the customer solution handled by IFS Lifecycle Experience	In the customer solution repository, in the form of exported configurations.	In the customer solution repository, in the form of models and code.

	Extend on the outside	Configure	Extend on the inside (Customize)
How it is applied, stored, and propagated.	Developed and deployed outside the control of IFS Lifecycle Experience.	Publish while application is running. Export and store in Customer Solution Repository. Ship through delivery. Import into other environments.	Checked into Customer Solution Repository. Built, delivered, installed into environments in Build Place and Use Place.

Combining tailoring types

Looking at a wider range of tailoring requirements for a customer solution, it is commonplace to use a combination of tailoring types, choosing the best tailoring type for each requirement.

When making these choices, please consider:

- **A customization should never have dependencies on configuration** as this would break the principles of the Layered Application Architecture (LAA) and risk causing deployment and installation failures. For example, don't make a customization containing business logic dependent on a custom attribute added using configuration. Instead add that attribute as part of the customization.
- **Minimize writing code (e.g. SQL, PLSQL) inside configurations.**
If your tailoring needs complex SQL queries or larger blocks of PLSQL, then it is recommended to implement the "code" parts as a customization, and then using a smaller/easier configuration on top, with dependency to the customization. This provides a better experience (a real code editor with intelligence, static code analysis, unit testing possibilities, ...) compared to "coding in a text field in a config tool" and also protects the investment from future deprecation and removal of "code" possibilities inside configurations.
- **Customizations can be used as "interfaces" to other parties, separating responsibilities**
Since the customizing by extending on the inside requires a different persona (developer) and additional tooling (IFS Developer Studio) than configuration or extend on the outside, it is natural and quite common that a higher skilled party creates a customization that serves as a simplified interface upon which another party can then create configurations. An example could be the first party creating a customization that encapsulate a complex SQL query behind a view, and then the second party uses to configure Lobby data sources and elements.

- **If a feature is part done using customization, consider doing it completely using customization**

Often what is perceived as a single feature/need for the customer is realized by tailoring several things, e.g. customizing some entity, overriding some function, changing a page layout. If a large part of the feature is realized through customization, then consider doing all of it (e.g. also the changes to page layouts) as customization even if it would be possible to do with configuration. This makes it easier to keep the entire feature together for tracking and delivery. Exceptions to this are tailorings such as lobby and report layouts that are only possible to do through configuration.

- **Never develop database artifacts as part of a configuration**

Database artifacts such as tables, views and PLSQL packages are not to be defined as part of a configuration (however some of those artifacts may be generated by the IFS Cloud platform when the configuration is published). If you see a need to use database artifacts to minimize the amount of code in a configuration they should be created as a customization (using the developer tooling and customization projects, i.e. the “cust” layer) upon which the configuration then depends. This is essential to assure things can be deployed to the use place in all deployment models (in particular in cloud), and to assure that current and future impact analysis tools “see” the database artifacts that are part of customer solution.

It can be tempting to “overuse” configurations because these are perceived to be easier or cheaper to do, e.g. by customer self-serving the work. But then remember that with IFS Cloud (unlike previous versions of IFS Applications) customers can self-serve development, use freely available tools, and handle everything consistently withing IFS Lifecycle Experience.

Guidance and principles for tailoring types

Configuration

Configuration refers to the use consultant and power-user friendly designers and tooling to extend the data model, change the user interface, automate processes and much more. Configuration is typically done directly by customers, or someone acting on behalf of the customer. Note that the use of application provided process and rule configuration, for the purpose of this tailoring guide, is considered part of the core application functionality rather than a configuration of the customer solution.

While it is easy to create and publish configurations in any environment, it's important that they too are treated and handled as a part of the complete customers solution.

Key principles for configurations:

- Configurations can be initially done / created in any environment, whether in the build place or use place. Most configurations have a built-in staging mechanism to prepare a

configuration and then “publish” it, which makes it active in that environment.

- Regardless of where configurations are initially created, they should always be exported and stored in the customer solution repository. This is essential for the ability to re-create the complete customer solution in new environments, for IFS Lifecycle Experience to provide impact analysis on configurations as part of assisted application of new releases, and for the ability to run current and future automated testing with the configurations present (making sure the true customer solution is tested).
- The mechanism to move configurations between environments is through inclusion in a delivery, which results in them being automatically imported when the delivery is installed. Using this delivery path for moving configurations is important so that the complete customer solution can be automatically recreated in new environments (in the build place) and used e.g. for automated testing. This type of usage will increase in the future. It is also possible to export and import manually to move them from one environment to another, but this is not recommended as the normal practice.

Important advice regarding configurations:

- Although some configuration capabilities (e.g. quick reports, custom event actions, lobby data sources) allow snippets of code (e.g. SQL) to be used within the configuration, it is strongly encouraged that any larger blocks of code are done as customizations using extend on the inside tailoring as this provides a better experience (a real code editor with intelligence, static code analysis, unit testing possibilities, ...) compared to “coding in a text field in a config tool” and also protects the investment from future deprecation and removal of “code” possibilities inside configurations.
- Follow the guidelines related to naming standards for configurations, where to place exported configurations in the solution repository etc. Adherence to the guidelines helps streamlines processes.
- Be mindful of the performance impact of adding configurations, as all added configuration can have an impact on performance of the application. This applies in particular to configurations (e.g. lobby data sources, quick reports) that define their own data queries, and to configurations that execute frequently (e.g. a process automation workflow attached to a high-volume transaction).
- Document configurations in order to describe their purpose, dependencies (on other configurations and/or on customizations) as well as any other information useful to share with people involved with the customer solution.

Extend on the outside

Extending on the outside refers to the use of IFS Cloud open RESTful APIs to integrate with, or build extensions to, the solution completely on the outside of the scope of IFS Cloud. Such extensions can be built using anything from low-code platforms like PowerApps or Mendix to plan programming in Java, .Net or any other language. Extending on the outside is typically

done by the customer or an third party providing development or integration services for the customer.

Key principles for extending on the outside:

- For IFS Cloud only API types oData APIs, Information Sources, and Events are to be used for extending on the outside.
- The APIs that are available for customers to use when extending on the outside are generally the same APIs that are used from the IFS Cloud user interface, which means that API coverage is complete in the way that anything you can do in IFS Cloud can be done through APIs.
- Things built using extend on the outside principles are not part of the IFS Cloud solution and should not be stored in the customer solution repository, nor are they to be managed using IFS Lifecycle Experience.

Important advice regarding extending on the outside:

- Use Premium APIs where available as these are better documented and more stable over time, resulting is less risk of breaking any usage of the API.
- Track and document your API usage so that you can assess the impact on that usage when updating to a new release. While IFS Lifecycle Experience may provide information about API changes in a release, it is not aware of the API usage and thus not able to assist in any impact analysis.
- If an API you are using is marked as deprecated, change to use another API as the deprecated API is likely to be removed in a future release.
- In special cases where the existing core APIs do not provide needed functionality new APIs can be created either as a projection configuration or by extending on the inside using a customized projection.

Extend on the inside

Extending on the inside refers to customization of the solution using the IFS Cloud platform and IFS Developer Studio to make additions or changes. This involved editing models, low-code development in the Marble domain specific language, as well as coding in other languages such as SQL, PLSQL, Java.

Extending on the inside is a powerful capability and is the only way certain things, such as overriding or changing business logic in the standard application, can be done. It is also the most invasive and spans a broad range from smaller additions that are easy to carry forward to future releases, to invasive changes that will result in additional work to analyze impact, uplift code etc. when applying future releases. Hence use the power of extending on the inside wisely, following the key principles and mindful of the advice.

Key principles for extending on the inside:

- Customizations are initially done / created in DEV environment in the build place, committed to the customer solution repository, and propagated to other environments in the build place through build pipelines.
- Propagation of customizations from the build place to the use place is always through the creation of a delivery, which is then installed in a use place environment.
- Never make customizations dependent on configurations as this risk causing deployment and installation failures. For example, do not make a customization containing business logic dependent on a custom attribute added using configuration. Instead add that attribute as part of the customization.
- Never customize anything in a platform components as this could impair the appropriate function of IFS Cloud. Also, other components could be set as `@final` for the same reason. This means they are locked from being customized. The full list of components can be found [here](#).
- Never mark a projection added or modified as part of a customization as being a premium API. The use of Premium API class is reserved for IFS. Future release updates may automatically remove premium API classification that has been added as part of a customization.
- Never include fragments from the core application layer into model files (page models, projection models etc.) in the customization layer, as these do NOT constitute an interface. IFS may change or remove fragments in the application core layer at any time and without prior notification.
- Do not customize or develop functionality for [IFS.ai Copilot](#).

Important advice regarding extending on the inside:

- Place any added objects (entities, projections, pages, ...) inside a new component dedicated for this purpose.
- Follow naming guidelines regarding things (components, pages, entities, attributes etc.) added as part of extending on the inside.
- Use `@overtake` as a last resort only since when you do this you effectively branch the standard code, creating your own copy of it inside your customization and taking responsibility for keeping this code updated with regards to any changes in the standard code in future releases or even service updates. If at all possible, use `@override` instead.

Tailoring Levels

The level of tailoring determines what options for tailoring are available in the tailoring experience as well as the expectations on the experience of updating to future releases. The lowest tailoring level is **core**, followed by **configured** and the highest level **customized**.

With the initial release of IFS Cloud, these tailoring levels are defined by guideline only, i.e. there is nothing that stops a customer who intends to be e.g. on the configured level from adding a customization. In the future the tailoring levels are likely to become more explicit and visible to customers as we continue to improve and streamline the experience for all levels.

	Core	Configured	Customized
Description	Staying as close to the standard software as possible to get the best experience (lowest efforts, lowest risks etc.) when updating to new releases. The tailoring experience is limited to personalization (e.g. individual user preferences such as bookmarks and saved searches) and parts of configuration that are closely related to how the user navigates in the application.	Staying close to the standard software to get a good experience when updating to new releases, but allowing the use of all configuration capabilities and extending on the outside using any APIs. This comes at the expense of a higher risk that some configurations could be impacted when applying future releases, and thus an expectation that updating to a new release will require a some effort to analyze the impact, potentially adjust configuration/integrations, and perform some testing. The level of effort needed will vary depending on the number and complexity of configurations and integrations used.	On the customized level the possibility to make additions or changes using the full developer toolset is added. Smaller customizations can be relatively easy to handle in terms of future release updates—often not more work than handling similar configurations. The larger and more intrusive (i.e. does overrides or overtakes to change functionality) customizations are more likely they are to be affected by future release updates and thus need to be adjusted.
Personas	End User Integration Developer	Power User Application Consultant	Application Developer - Low-code using

	Core	Configured	Customized
		Integration Developer	IFS DSL - Full code
Tailoring experience	Personalization—user preferences Designers—Lobby, Navigator Application provided process and rule configuration Extend on outside using Premium APIs	All designers—Lobby, Navigator, Page, Workflow etc. Configuration Tools inside IFS Solution Manager Extend on the outside using all APIs	In addition to Configuration: Extend on inside using IFS Developer Studio a) Extend with new functionality using IFS Cloud Platform b) Change existing functionality with Layered Application Architecture (LAA) Configuration Tools that include option to add in “code” e.g. in the form of SQL
Release update experience	Automated using IFS Lifecycle Experience Automated update of data, application provided configuration, and personalization	Impact analysis supported by IFS Lifecycle Experience Automated update of data, application provided configuration, and personalization Assisted update of impacted configurations	Developer Guidance on how to address occasional changes to Platform Assisted update of impacted overridden functionality a) Added functionality continues to work

Core	Configured	Customized
		b) Assisted merge/uplift of added functionality

Common scenarios

The below table describe some common tailoring needs along with recommended and discouraged options.

Scenario	Recommended	Possible	Discouraged
I want to add a new field that should be stored and shown inside existing pages in the UI and/or report layouts	If it is just about showing and storing the information, configure a custom attribute and add to the UI and reports using Page Designer / Report Designer	Extend on the inside to override the entity, pages and report definitions to add in the attribute/field	
I have custom menu items implemented in PLSQL in Enterprise Explorer and need to upgrade these to IFS Cloud Web	Customize the Projection of the underlying page in IFS Cloud Web adding a new action containing the PLSQL logic. Then use this new action in a Custom Command added from the Page Designer	Customize a new Utility Package, adding various PLSQL Methods. Then add a new Custom Action on the Projection.	

Scenario	Recommended	Possible	Discouraged
I have custom event actions implemented as Execute Online SQL that run complex code, perhaps held in utility APIs, that depend on custom attributes and other configurations	IFS is working on phasing out these capabilities. Therefore the recommendation is to replace these kinds of solutions with Workflows, Query Designer etc.	Achieve these requirements using Customization (Extending on the Inside)	Continue to use Execute Online SQL event actions and Utility packages